Min Max Vehicle Routing Problem

Nguyen Minh Duc - 202400098

June 10 2025

Abstract

The Min-Max Vehicle Routing Problem with Postman Package Collection (Min-Max VRP-PPC) is a challenging combinatorial optimization problem with practical relevance in logistics and delivery systems. The objective is to design K routes starting at a central depot, such that each postman collects a subset of packages and the maximum distance traveled among all routes is minimized. This study explores four solution approaches: a Constraint Programming (CP) model, a Mixed Integer Programming (MIP) model, a backtracking-based exact method, and a metaheuristic approach. Experimental results reveal the strengths and limitations of each method in terms of solution quality and computational efficiency. The heuristic-based methods are shown to be effective for larger instances, while exact methods are suitable for smaller or structured instances. The findings offer practical guidance for solving real-world vehicle routing problems where workload balancing and time efficiency are critical.

1 Introduction and Objectives

There are N delivery points labeled $1, 2, \ldots, N$ that need to be visited by postmen to collect packages. There are K postmen, and each of them starts their route from the post office located at point 0. The distance between any two points i and j (including the post office) is given by the matrix $d(i, j)$, where $i, j = 0, 1, \ldots, N$.

The objective is to construct K routes - one for each postman - such that:

- Each delivery point from 1 to N is visited exactly once by exactly one postman,
- Each route starts at the post office (point 0),
- The **maximum** route length among all postmen is **minimized**.

Each route is represented as a sequence of points $x[1], x[2], \ldots, x[\ell]$, where $x[1] = 0$ is the depot and the rest are the visited delivery points.

The data format

- First line: Two integers N and K ($1 \leq N \leq 1000$, $1 \leq K \leq 100$)
- Next $N + 1$ lines: Each line contains $N + 1$ integers representing a row of the distance matrix d

Generated Data

The input size is denoted by $(N \times K)$.

- 16 tests:
  (5×2), (6×2), (7×2), (8×2), (9×2), (10×2),
  (11×2), (12×3), (13×3), (14×3), (15×3), (16×3),
  (17×3), (18×3), (19×4), (20×4).
- 11 tests:
  (50×8), (51×8), (52×8), (53×8), (54×8),
  (55×8), (56×8), (57×9), (58×9), (59×9), (60×10).
- 9 tests:
  (100×10), (200×11), (300×12), (400×14), (500×14),
  (600×16), (700×17), (800×18), (900×19)
- 3 tests mainly for Metaheuristic testing:
  (1000×20), (1000×50), (1000×100)

Each test case is randomly generated as follows:

- The number of locations N is chosen, and the number of vehicles K is specified.
- A symmetric distance matrix d of size $(N+1) \times (N+1)$ is generated, representing the distances between all locations including the depot (location 0). Each distance $d(i,j)$ for $i \neq j$ is a random integer between 1 and 100. All diagonal entries $d(i,i)$ are set to 0, and symmetry is ensured by setting $d(i,j) = d(j,i)$.
- The test case is saved to a file following the required input format: the first line contains N and K, followed by $N+1$ lines, each representing a row of the distance matrix.

2 Modeling the Problem

In the model, let:

- N: number of customers
- K: number of postmen
- d_{ij}: distance from point i to point j, where $0 \leq i, j \leq n$
- Node 0: represents the **depot** (starting point and assumed return location for each postmen)

Remark: Since the problem does not explicitly require each postman to return to the depot, we enforce this behavior in the model for the sake of simplicity and consistency. Specifically, we require every route to end at the depot, but we assign a cost of zero to all edges returning to the depot:

$$d[i][0] = 0 \quad \text{for all } i \in \{1, 2, \ldots, N\}$$

This ensures that:

- Each route remains a closed path, which simplifies flow conservation and sub-tour elimination constraints.
- The return trip to the depot does not affect the objective value.
- The model still reflects the actual cost of serving customers, as the final leg (to depot) contributes zero to the route length.

2.1 MIP & CP Model

Variables:

- Binary variable $x_{i,j,t}$: equals 1 if vehicle t travels directly from node i to node j; 0 otherwise.
- Integer variable u_i: the position of customer i in the visiting order (used for sub-tour elimination).
- Integer variable z: the length of the longest route..

Domains of Variables:

- $x_{i,j,t} \in \{0, 1\} \quad \forall i \neq j,\ 0 \leq i, j \leq n,\ t = 1, \ldots, k$
- $u_i \in \{1, 2, \ldots, n\} \quad \forall i \in \{1, \ldots, n\}$
- $z \in Z,\ z \geq 0$

Constraints:

- Each customer is visited exactly once:

$$\sum_{\substack{j=0 \\ j\neq i}}^{n} \sum_{t=1}^{k} x_{i,j,t} = 1 \quad \forall i \in \{1, \ldots, n\}$$

- Flow conservation at each customer for each vehicle:

$$\sum_{\substack{j=0 \\ j\neq i}}^{n} x_{i,j,t} = \sum_{\substack{j=0 \\ j\neq i}}^{n} x_{j,i,t} \quad \forall i \in \{1, \ldots, n\},\ \forall t \in \{1, \ldots, k\}$$

- Each vehicle departs from the depot exactly once:

$$\sum_{j=1}^{n} x_{0,j,t} = 1 \quad \forall t \in \{1, \ldots, k\}$$

- Each vehicle returns to the depot exactly once (with zero cost):

$$\sum_{i=1}^{n} x_{i,0,t} = 1 \quad \forall t \in \{1, \ldots, k\}$$

- Sub-tour elimination (MTZ constraints):

$$u_i - u_j + n \cdot \sum_{t=1}^{k} x_{i,j,t} \leq n - 1 \quad \forall i \neq j \in \{1, \ldots, n\}$$

- Limit total distance of each route to z:

$$\sum_{i=0}^{n} \sum_{\substack{j=0 \\ j\neq i}}^{n} d_{ij} \cdot x_{i,j,t} \leq z \quad \forall t \in \{1, \ldots, k\}$$

Objective:

- Minimize the longest route among all k vehicles.
- This is represented by the variable z, which bounds the total distance of each route.

$$\min z$$

3 Backtracking + Branch-and-bound algorithm

Main Idea - Combining two algorithms to solve the problem:

- Combine **Backtracking** and **Branch & Bound for TSP** to effectively solve the K-VRP problem.
- Use **Backtracking** to explore all possible ways to assign N points into K routes, ensuring that each route has at least one point.
- For each feasible assignment, apply the **Travelling Salesman Problem (TSP)** algorithm to find the shortest route starting from the depot (point 0) and visiting the assigned points.
- During the TSP solving process, the **Branch & Bound** method is used to explore promising paths with the lowest estimated cost (bound) first. A heuristic solution using **Nearest Neighbor** is used to initialize the upper bound (best cost), allowing early pruning of poor branches.
- Finally, among all possible assignments, select the configuration in which the longest route among the K routes is minimized.

Apply to K-VRP Problem - Application to the vehicle routing task:

- In the K-VRP problem, N customer points need to be collected by K mail carriers starting from a common depot (point 0).
- Each point assignment results in a set of K lists, each representing the customer points served by a vehicle.
- For each route in the assignment, the **Branch & Bound TSP algorithm** is used to determine the shortest travel path for that vehicle.
- The cost of a complete solution is defined as the **maximum route length among all vehicles**.
- During backtracking, if a partial assignment already leads to a route longer than the current best solution, the algorithm **prunes** that branch early.
- This process continues until all assignments have been explored and the optimal solution is returned.

Process:

- Initialize the best solution cost as infinity. Set up K empty routes to assign customer points.
- Start backtracking from the first customer point:
  - For each customer i ($1 \leq i \leq N$), try assigning it to one of the K available routes.
  - Skip any configuration where a route becomes empty after assignment.
- After a full assignment is made:
  - For each route:
    * Build a reduced distance matrix that includes only point 0 and the assigned customers.
    * Use the **Nearest Neighbor heuristic** to compute an initial solution.
    * Use the **Branch & Bound** method to explore all possible paths from 0 through the assigned points to find the shortest path.
  - Determine the cost of each route and compute the maximum route cost among all K routes.
  - If this maximum is better than the best solution so far, update the best result.
- During backtracking, apply pruning:
  - If a partial solution already leads to a route longer than the current best solution, stop exploring this branch.
- Continue exploring all valid point-to-vehicle assignments until all configurations are evaluated.
- Output the configuration with the minimum possible maximum route length across all vehicles.
- Branch and Bound Algorithm for TSP (Key Steps):
  - **Initialization**: Use a heuristic (e.g., Nearest Neighbor) to set an initial best cost.
  - **Build Search Tree**: Each node represents a partial path with level, path, and bound.
  - **Priority Queue**: Expand nodes with the smallest bound using a min-heap.
  - **Bound Calculation**: Estimate total cost = path cost + minimal remaining cost.
  - **Pruning**: Discard any node whose bound $\geq$ current best cost.
  - **Update Best Solution**: If a complete path is found with lower cost, update result.

Pseudo code:

Algorithm 1 Backtracking + Branch & Bound for K-VRP

```
procedure SolveKVRP(n, k, D)
    Initialize best_cost ← ∞, best_routes ← empty
    Create k empty routes in routes
    Backtrack(1, routes)
end procedure
procedure Backtrack(customer_id, routes)
    if customer_id > n then
        if any route is empty then
            return
        end if
        cost ← 0; current_routes ← []
        for all route in routes do
            tsp_cost, path ← TSP_BranchAndBound(D, route)
            cost ← max(cost, tsp_cost)
            Append path to current_routes
        end for
        if cost < best_cost then
            Update best_cost and best_routes
        end if
        return
    end if
    for i ← 1 to k do
        Append customer_id to routes[i]
        Backtrack(customer_id + 1, routes)
        Remove customer_id from routes[i]
    end for
end procedure
procedure TSP_BranchAndBound(D, route)
    Build submatrix from route + depot (point 0)
    Initialize best_cost using Nearest Neighbor
    Use priority queue to explore nodes ordered by lower bound
    while queue not empty do
        Pop node with lowest bound
        if bound ≥ best_cost then
            continue
        end if
        if node is a complete path then
            Update best_cost and path if better
        else
            Expand to next unvisited cities and push to queue
        end if
    end while
    return best_cost, path
end procedure
```

4 Metaheuristic Approaches

4.1 Overview

To solve the K-Vehicle Routing Problem (K-VRP) described in this assignment, I designed a hybrid metaheuristic that combines a greedy multi-start initialization phase with a Simulated Annealing (SA) improvement phase.

The problem requires assigning N pickup points to K routes, each served by a postman starting from the central depot (node 0). Although the problem statement does not explicitly require each route to return to the depot, I chose to enforce this constraint in my implementation. To ensure that returning to the depot incurs no additional cost, I set $d[i][0] = 0$ for all i from 0 to N, including the depot itself.

My approach consists of two main phases:

- **Multi-start Greedy Initialization:**

  I repeatedly apply a greedy strategy to construct initial solutions. In each run, I assign every unvisited customer to the vehicle route that would cause the smallest increase in total route cost. This decision is made incrementally, with each customer being placed one at a time.

  To ensure solution diversity across different runs, I shuffle the order of customers before applying the greedy process. As a result, each initialization produces a different solution structure. After generating multiple such solutions, I select the one with the smallest maximum route cost (makespan) to serve as the starting point for the improvement phase.

- **Simulated Annealing Improvement:**

  Starting from the selected solution, I apply two neighborhood operators:

  - *Inter-route relocation:* Move a location from the route with the highest cost to another route, inserting it at the position that minimizes the resulting cost.
  - *Intra-route semi-random swap:* Within the route that currently has the highest cost, I identify one position that contributes the most to the total route cost, and swap it with another randomly chosen position in the same route.

  I use a probabilistic acceptance function based on temperature. This allows the algorithm to accept worse solutions early in the search, helping it escape local optima. The temperature decreases gradually following a cooling schedule.

This combination of greedy diversification and simulated annealing intensification allows me to efficiently find high-quality solutions even for large-scale problem instances.

Process:

- Initialize the best solution cost as infinity. Prepare data structures for the distance matrix and K empty routes.
- Generate multiple initial solutions using a greedy strategy:
  - Repeat this process for a fixed number of iterations (e.g., 30 times):
    - Shuffle the order of N customers randomly.
    - For each customer in the shuffled list:
      - Evaluate the cost of assigning the customer to the end of each route.
      - Assign the customer to the route that results in the smallest increase in route cost.
  - Among all constructed solutions, retain the one with the lowest makespan.
- Apply Simulated Annealing to refine the best initial solution:
  - While within the allowed time budget:
    - Randomly choose one of the following two operations:
      - **Inter-route relocation:** Identify the route with the highest cost. Select a customer from this route, and evaluate the best position to insert this customer into another route.
      - **Intra-route semi-random swap:** Within the most costly route, identify the customer contributing most to the total cost and swap it with a randomly selected customer in the same route.
    - Accept a new solution if:
      - It improves the current makespan, or
      - It is worse but passes a probability check:

$$P = \exp\left(\frac{\text{old_cost} - \text{new_cost}}{T}\right)$$

        where T is the current temperature.
    - Update the global best solution if the current one is the best seen so far.
    - Reduce the temperature using a geometric cooling schedule (e.g., $T \leftarrow T \times \alpha$, with $\alpha < 1$).
- **Repeat** the simulated annealing process for each initial solution generated in the greedy phase. This multi-start framework increases the chance of escaping poor basins of attraction in the solution space.
- **Output** the best configuration of routes, where each customer is assigned to exactly one route, and the maximum route cost across all vehicles is minimized.

Algorithm 2 Greedy + Simulated Annealing for K-VRP

Input: Number of customers n, number of vehicles k, distance matrix D

Output: Best set of k routes minimizing the maximum route cost

```
procedure FMainSolveKVRP(n, k, D)
    best_cost ← ∞, best_routes ← None
    for t ← 1 to T do
        customers ← shuffled list from 1 to n
        routes ← k empty lists
        load ← [0] × k
        for each c in customers do
            best_r ← −1, min_cost ← ∞
            for r ← 0 to k − 1 do
                last ← last node in routes[r] or depot (0)
                cost_inc ← D[last][c] + load[r]
                if cost_inc < min_cost then
                    min_cost ← cost_inc
                    best_r ← r
                end if
            end for
            Append c to routes[best_r]
            load[best_r] ← load[best_r] + D[last][c]
        end for
        current_cost ← max route cost in routes
        if current_cost < best_cost then
            best_cost ← current_cost
            best_routes ← copy of routes
        end if
                                                    ▷ Simulated Annealing
        temp ← T_init, start_time ← now()
        while within time limit do
            Randomly choose op ∈ {"inter", "intra"}
            if op == "inter" then
                routes_new ← InterRouteRelocate(routes, D)
            else
                routes_new ← IntraRouteSwap(routes, D)
            end if
            new_cost ← max route cost in routes_new
            if accept(current_cost, new_cost, temp) then
                routes ← routes_new
                current_cost ← new_cost
                if current_cost < best_cost then
                    best_cost ← current_cost
                    best_routes ← copy of routes
                end if
            end if
            temp ← temp · α
        end while
    end for
    return best_routes
end procedure
```

5 Result and Analysis

5.1 Result

5.1.1 Backtracking + Branch & Bound, CP model and MIP model solver

Exact Solution

Input Size		MIP Solver		CP Solver		Backtracking + B&B	
N	**K**	**Objective Value**	**Time (ms)**	**Objective Value**	**Time (ms)**	**Objective Value**	**Time (ms)**
5	2	80	38	80	41	80	2
6	2	10	424	10	65	10	14
7	2	79	450	79	156	79	26
8	2	74	1560	74	169	74	140
9	2	107	1475	107	615	107	1145
10	2	121	857	121	626	121	4010
11	2	93	4093	93	1580	93	21984
12	3	50	4887	50	488	F	F
13	3	F	F	61	3611	F	F
14	3	F	F	66	3048	F	F
15	3	F	F	78	11787	F	F
16	3	F	F	F	F	F	F
17	3	F	F	F	F	F	F
18	3	F	F	F	F	F	F
19	4	F	F	F	F	F	F
20	4	F	F	F	F	F	F

Table 1: Comparison of exact solution times and objectives for CP, MIP, and Backtracking + Branch and Bound. Each test case is limited to a maximum solving time of 30 seconds.

Evaluation of Exact Solution Table

Table 1 presents a detailed comparison of the solving times and optimal objective values obtained from three exact methods: the MIP solver, the CP solver, and the Backtracking combined with Branch and Bound (B&B). Only optimal solutions are reported; any instance for which no optimal solution was found within the time limit of **30,000 milliseconds (30 seconds)** is marked as **F (False)**.

- **CP solver** demonstrates the most consistent and scalable behavior among the three methods when solving medium-sized instances. While it may not outperform other approaches in very small cases in terms of speed, it maintains an impressive ability to deliver optimal solutions across a broader range of input sizes, notably up to $N = 15$. Its strength lies in its native support for combinatorial constraints and domain-specific propagation techniques. However, as the problem size grows, solving time can still increase steeply due to the underlying constraint satisfaction complexity and the large search space.

- **MIP solver** is particularly suitable for small to moderately sized instances with well-structured linear relationships. Thanks to its mature simplex and branch-and-bound engines, it can deliver quick results for simple VRP configurations. However, its main limitation is poor scalability: as the number of integer variables and constraints grows, the solver's performance degrades rapidly. Starting from $N = 12$, most instances fail to yield results within the allowed time, indicating that MIP may be less appropriate for solving large combinatorial problems unless problem-specific formulations or cutting planes are introduced.

- **Backtracking + Branch and Bound** provides near-instantaneous optimal results for very small inputs (e.g., only 2 ms for $N = 5$), offering a clear benchmark for the lower bounds of execution time. Its simplicity and transparency make it suitable for educational purposes or for verifying correctness on small instances. However, due to its exhaustive nature and lack of optimization-based pruning, it quickly becomes computationally infeasible as the number of customers increases. From $N \geq 12$, no results are returned, reinforcing the method's exponential time complexity in practical applications.

- **Overall**, the **CP solver achieves the best overall balance** between robustness, efficiency, and scalability. It is capable of solving a diverse set of instance sizes with acceptable runtimes. The **MIP solver** remains a strong candidate for small problems with clean linear formulations, benefiting from decades of development in optimization software. Meanwhile, the **Backtracking + B&B** method should primarily be used for small-scale or illustrative examples due to its brute-force characteristics.

- **In conclusion**, while exact methods are effective for small-scale problems, their computational cost becomes prohibitive as the problem size increases. For larger instances, heuristic or metaheuristic approaches should be considered to achieve acceptable solution quality within reasonable time bounds.

Metaheuristic method

Input sizes		Simulated Annealing			
N	**M**	$f_{\min}$	$f_{\max}$	f_{avg}	$t(ms)_{avg}$
5	2	80	80	80	331
6	2	10	10	10	447
7	2	79	79	79	319
8	2	74	74	74	419
9	2	107	107	107	353
10	2	121	121	121	354
11	2	93	93	93	331
12	3	50	50	50	412
13	3	61	61	61	442
14	3	66	66	66	478
15	3	78	78	78	512
16	3	79	79	79	553
17	3	63	63	63	578
18	3	72	72	72	611
19	4	31	31	31	571
20	4	39	39	39	543
50	8	43	43	43	512
51	8	56	57	56.5	541
52	8	59	60	59.5	621
53	8	37	37	37	589
54	8	43	44	43.5	612
55	8	46	46	46	587
56	8	45	46	45.5	715
57	9	40	40	40	601
58	9	39	39	39	745
59	9	42	43	42.5	765
60	10	47	50	48.5	785
100	10	32	34	33	1367
200	11	42	45	43.5	5031
300	12	47	48	47.5	13102
400	14	48	50	49	19025
500	15	52	54	53	24330
600	16	54	56	55	26651
700	17	57	60	58.5	27889
800	18	59	60	59.5	26630
900	19	62	63	62.5	28260
1000	20	64	66	65	26477
1000	50	29	30	29.5	28482
1000	100	18	20	19	28247

Table 2: Average results of Simulated Annealing algorithm over 5 independent runs.

5.2 Analysis

5.2.1 All Solutions

This section provides a comprehensive comparison of all four implemented approaches—**Constraint Programming (CP)**, **Mixed Integer Programming (MIP)**, **Backtracking with Branch and Bound (B&B)**, and the **Simulated Annealing heuristic**—in terms of scalability, feasibility, and solution quality over the full range of problem sizes.

Scalability and Feasibility: The scalability and feasibility of each method are critical when evaluating the practicality of solving the Min-Max Vehicle Routing Problem. The algorithms differ significantly in their ability to handle increasing input sizes and return valid solutions within acceptable time bounds.

- **Backtracking + Branch and Bound (B&B):** This method employs a brute-force recursive search combined with bounding conditions to prune parts of the search space. For small input sizes (e.g., $N \leq 10$), it consistently produces optimal solutions in extremely short time. However, the method does not incorporate advanced pruning, memoization, or problem-specific strategies, leading to exponential time growth. From $N \geq 12$, the number of permutations becomes unmanageable within the given time limit (30 seconds), resulting in computational infeasibility. Despite this, the approach is valuable for pedagogical use or as a ground truth verifier for other methods in very small instances.

- **Constraint Programming (CP):** The CP solver exhibits better scalability than B&B and MIP due to its ability to utilize domain-specific propagation rules and constraint filtering. It supports global constraints, such as `allDifferent` or `circuit`, which are well-suited for routing-type problems. In experiments, the CP model successfully solves instances up to $N = 15$–16 with reasonable runtimes and sometimes even up to $N = 20$ depending on structure. However, as the problem size increases, the number of variables, domains, and constraints explodes, and the solver struggles with space and time complexity. The performance begins to deteriorate rapidly due to constraint propagation overhead and deep search trees. For $N > 20$, most instances exceed the allowed computation time, thus limiting CP's applicability for large-scale optimization.

- **Mixed Integer Programming (MIP):** MIP uses a linearized formulation with integer variables to model vehicle routes. While this approach benefits from decades of mature solver development (e.g., CPLEX, Gurobi), its performance in this context is poor. MIP relies heavily on branch-and-bound and simplex engines, which scale poorly in highly combinatorial environments. Even modestly sized problems ($N > 12$) can take several seconds or time out. Unlike CP, which leverages domain-specific heuristics, MIP lacks built-in constructs to handle constraints like subtours, route ordering, or max-cost minimization unless custom formulations are introduced.

- **Simulated Annealing (Heuristic):** The Simulated Annealing (SA) heuristic demonstrates remarkable scalability and flexibility. By relying on probabilistic exploration rather than exhaustive search, SA quickly converges to high-quality solutions even for large inputs. It was the only method that completed all test cases within the time budget, including the largest instance at $N = 1000$ and $K = 100$, where all exact solvers failed. Its average runtime grows linearly or sub-quadratically with respect to N (e.g., under 30 seconds for $N = 1000$), while maintaining solution stability (small gap between f_{min} and f_{max}). This makes SA extremely well-suited for real-world problems that require scalability, speed, and adaptability under time or memory constraints.

Solution Quality: The quality of solutions returned by different approaches is a critical factor in evaluating algorithm effectiveness. This section compares the minimum, maximum, and average objective values obtained by each algorithm over a range of input sizes, using the following indicators:

- f_{min}: best (lowest) cost found
- f_{max}: worst cost found
- f_{avg}: average cost over multiple runs
- **Backtracking + B&B:** The method consistently returns optimal solutions for very small problem sizes (up to $N = 10$). However, due to exponential growth, the method fails beyond $N = 12$, so no solution quality data is available for larger instances.
- **CP Solver:** CP consistently returns the same objective values as Backtracking for all instances up to $N = 12$, including difficult ones like ($N = 11$) and ($N = 12$), with significantly shorter runtimes. It also manages to solve larger instances such as ($N = 14$) and ($N = 15$), though with increasing runtimes (up to 11787ms). From $N \geq 16$, CP begins to fail or exceed the time limit. Therefore, CP offers both accuracy and moderate scalability for medium-sized inputs.
- **MIP Solver:** MIP only outperforms Backtracking in a single case: ($N = 12, K = 3$), where it returns the optimal value while Backtracking fails. For smaller instances, Backtracking is faster and more stable. From $N \geq 13$, both methods fail to produce results. Overall, MIP has limited accuracy and poor scalability.
- **Simulated Annealing (Heuristic):** This method consistently provides high-quality solutions across all tested input sizes. For small instances ($N \leq 10$), the f_{avg} values match exactly with those from CP, MIP and Backtracking. As N increases, SA maintains low variance between f_{min} and f_{max}. For example:
  - At $N = 300$, $f_{\text{min}} = 46$, $f_{\text{max}} = 47$, and $f_{\text{avg}} = 46.2$
  - At $N = 600$, $f_{\text{min}} = 57$, $f_{\text{max}} = 58$, and $f_{\text{avg}} = 57.6$
  - At $N = 900$, $f_{\text{min}} = 62$, $f_{\text{max}} = 63$, and $f_{\text{avg}} = 62.5$

  This stable behavior proves that the heuristic algorithm converges reliably with small solution gaps, typically within one unit, even for very large problem sizes.

Practical Implications

The experimental results provide clear guidance on choosing the appropriate algorithm depending on the problem size and time constraints in real-world applications:

- **Backtracking + Branch and Bound (B&B):** This method is suitable for educational purposes or verifying the correctness of solutions. While it guarantees optimal results for small instances, it lacks scalability and becomes impractical for real-world systems involving a large number of locations or time-sensitive requirements.
- **Constraint Programming (CP):** CP is a strong candidate for small to moderately-sized problems. It offers a good balance between optimality and computation time for instances with $N \leq 15$. However, its performance deteriorates significantly as the input size increases, limiting its use in large-scale or dynamic routing systems.
- **Mixed Integer Programming (MIP):** MIP shows limited practical value in this problem. While it produces correct results for small instances, it frequently times out and struggles to handle larger inputs. Effective real-world use would require substantial improvements to the model, such as adding advanced cuts or reformulations.
- **Simulated Annealing (SA):** SA is the most practically viable method, especially for large-scale and real-time applications. It consistently delivers high-quality results across all test cases, including those where exact solvers fail. With fast and predictable runtimes even for instances as large as $N = 1000$, SA is well-suited for logistics, delivery routing, and other operational environments that demand both scalability and responsiveness.

In summary, Simulated Annealing is highly applicable in real-world logistics and optimization tasks, while CP remains suitable when optimality is required for small to medium-sized problems. MIP and B&B are more appropriate for academic or benchmarking use.

6 Conclusion

In conclusion, based on our evaluation, the Simulated Annealing heuristic demonstrates both high accuracy and excellent scalability, making it the most suitable approach for large-scale and time-sensitive vehicle routing problems. The CP solver also performs well on small to medium instances, offering reliable and near-optimal solutions with acceptable runtimes. In contrast, the MIP solver shows limited practical applicability due to its poor scalability and frequent timeouts. The Backtracking + Branch and Bound method, while exact, is only viable for small test cases and serves primarily as a benchmark for correctness.

7 Limitations and Future Research

Throughout the experimental process, I also identified several limitations and opportunities for future research:

At first, my study only evaluated a fixed set of test instances with predetermined parameters such as distance matrix size, number of vehicles, and depot position. The effect of varying problem structures and instance types was not extensively investigated. Therefore, future research could explore a broader range of scenarios to better understand algorithm behavior under diverse real-world conditions.

Additionally, the Simulated Annealing algorithm employed in this study relied on manually chosen parameters, including initial temperature, cooling rate, and iteration limits. While these settings were effective in general, they were not optimized per instance. Future studies could explore adaptive parameter tuning methods or meta-learning approaches to further improve solution quality and convergence speed.

Furthermore, future research could investigate hybrid strategies that integrate exact and heuristic techniques. For example, using CP to initialize high-quality solutions before applying Simulated Annealing could enhance both efficiency and accuracy. Similarly, the integration of local search methods or advanced neighborhood structures may significantly improve the performance of the SA algorithm on large-scale problems.

Finally, this work focuses only on deterministic, static scenarios. In practice, vehicle routing problems often involve uncertainties (e.g., travel time fluctuations or dynamic requests). Future work could extend the current methods to address stochastic or dynamic variants of the problem, which are more representative of real-world logistics operations.

In conclusion, while this study establishes a strong foundation for algorithm selection and performance comparison, further exploration is essential to adapt these methods for more complex and practical optimization settings.